

Exceptions Interview Questions

C++ Exception Handling — ամբողջական ժողովածու

Այս ֆայլը exception-neri interview հարցերի zhoxovacu e. Erek mas.

Մաս A — 30 multiple-choice. Մաս B — 25 bac (open) harc. Մաս C — 15 tricky code / "inch klini".

Bolor patasxannery **miayn verjum** en (Answer Key A, B, C).

ՄԱՍ A — Multiple Choice (30)

Q1

Exception inch e? - (a) memory leak - (b) error handling mekanizm (throw/try/catch) - (c) thread - (d) pointer

Q2

throw inch e anum? - (a) catch - (b) exception nety - (c) function kanchum - (d) return

Q3

try block inchi hamar? - (a) loop - (b) kod vor karo e throw anel (pahpanvac) - (c) thread - (d) lock

Q4

catch-i kargy? - (a) patahakan - (b) verevic var (konkretic general) - (c) varic verev - (d) chka karg

Q5

catch(...) inch e? - (a) syntax error - (b) catch-all (amen inch) - (c) miayn int - (d) rethrow

Q6

Stack unwinding inch e? - (a) stack overflow - (b) throw-i zham het call stack, local dtor-ner kanchvum - (c) memory azatum - (d) thread

Q7

Stack unwinding-i zham inch e kanchvum? - (a) constructor-ner - (b) local object-neri destructor-ner - (c) main - (d) ochinch

Q8

catch by value-i problem? - (a) arag - (b) slicing + copy - (c) leak - (d) ochmi

Q9

Chisht catch dzev? - (a) catch(exception e) - (b) catch(const exception& e) - (c) catch(exception* e) - (d) catch(exception&& e)

Q10

Destructor-ic throw anely? - (a) OK - (b) vtangavor - std::terminate (unwinding-i zham) - (c) arag - (d) leak

Q11

std::exception-i himnakan metody? - (a) message() - (b) what() - (c) error() - (d) msg()

Q12

vector::at() out of range -> ? - (a) UB - (b) throw std::out_of_range - (c) nullptr - (d) 0

Q13

new anhajoxutyun -> ? - (a) nullptr - (b) throw std::bad_alloc - (c) crash - (d) 0

Q14

noexcept inch e nshanakum? - (a) chi throw - (b) misht throw - (c) noexcept function - (d) atomic

Q15

noexcept function throw anelis? - (a) OK - (b) std::terminate - (c) catch - (d) return

Q16

Custom exception inchpes? - (a) throw int - (b) ja'angel std::exception, override what() - (c) throw string - (d) chkareli

Q17

Strong exception guarantee? - (a) chka leak - (b) throw-i depqum vichak chi poxvum (rollback) - (c) erbek throw - (d) crash

Q18

No-throw guarantee? - (a) noexcept - erbek chi throw - (b) rollback - (c) basic - (d) leak

Q19

RAII-i kapy exception-i het? - (a) ochmi - (b) exception-safe cleanup (unwinding -> dtor) - (c) arag - (d) leak

Q20

std::out_of_range vory-i tak e? - (a) runtime_error - (b) logic_error - (c) bad_alloc - (d) exception
unix

Q21

Rethrow inchpes? - (a) throw e; - (b) throw; (afanc argument) - (c) return - (d) catch

Q22

Exception normal control flow-i hamar? - (a) ayo, misht - (b) voch (danak e, dandaghn) - (c) miayn loop - (d) misht

Q23

catch(const exception&) bñnum e bad_alloc? - (a) voch - (b) ayo (bad_alloc jařangum exception-ic) - (c) miayn runtime - (d) CE

Q24

throw-ic heto kody? - (a) katarvum - (b) chi katarvum (control catch e gnum) - (c) return - (d) loop

Q25

Exception-i overhead (chka throw)? - (a) mec - (b) grethe zero (zero-cost, erb chka throw) - (c) 2x - (d) atomic

Q26

Multiple catch - vory-n e b́rnvum? - (a) verjiny - (b) afajin hamapatasxan (verevic) - (c) bolory - (d) patahakan

Q27

logic_error vs runtime_error? - (a) nuyny - (b) logic (kanxateseli, programming), runtime (ashxatanki) - (c) logic aveli vat - (d) chka tarberutyun

Q28

what() -i return type? - (a) string - (b) const char* - (c) int - (d) void

Q29

Constructor-ic throw anelis object-y? - (a) steghcvm - (b) chi steghcvm (dtor chi kanchvum ir hamar) - (c) leak - (d) crash

Q30

Exception-ы vs error code? - (a) nuyny - (b) exception `avtomat propagate, chi karoх anteswel; error code` dzérqov stugum - (c) error code aveli lav misht - (d) chka tarberutyun

ՄԱՍ B — Open Questions (25)

1. Exception inch e, inchu e petq.
2. try/catch/throw inchpes en ashxatum.
3. Stack unwinding manramasn.
4. Inchu RAII-y exception-safe e.
5. catch by value vs by reference.
6. Inchu destructor-ic chi kareli throw.
7. std::exception hierarchia.
8. what() metody.
9. Custom exception inchpes es sarqum.
10. Exception safety guarantees (4 makardak).
11. noexcept inch e, inchu e petq.
12. noexcept move constructor-i karevorutyuny (STL).
13. catch(...) inchpes u erb.
14. Multiple catch - kargy.

15. Rethrow (throw;).
 16. Exception vs error code.
 17. Exception-i overhead (zero-cost model).
 18. bad_alloc, bad_cast, out_of_range.
 19. logic_error vs runtime_error.
 20. Constructor-ic throw - object-i vichaky.
 21. Inchu exception normal flow-i hamar chi.
 22. Exception neutrality inch e.
 23. try-catch-i performance uqdecutyun.
 24. Exception-ner thread-neri mijev (chen propagate).
 25. Function-try-block inch e.
-

ՄԱՍ C — Tricky Code (15)

C1

```
try { throw std::runtime_error("x"); }  
catch (const std::exception& e) { cout << "caught"; }
```

- (a) caught (runtime_error ja'angum exception)
- (b) chi brnvi
- (c) CE
- (d) terminate

C2

```
try { throw 42; }  
catch (const std::exception& e) { cout << "ex"; }
```

- (a) ex
- (b) terminate (int chi ja'angum exception, chka catch)
- (c) 42
- (d) OK

C3

```
struct A { ~A(){ cout << "~A "; } };  
void f() { A a; throw std::runtime_error("x"); }  
try { f(); } catch(...) { cout << "caught"; }
```

- (a) caught
- (b) ~A caught (stack unwinding -> dtor)
- (c) caught ~A
- (d) terminate

C4

```
void f() noexcept { throw std::runtime_error("x"); }  
f();
```

- (a) caught
- (b) std::terminate (noexcept + throw)
- (c) OK
- (d) return

C5

```
try { throw std::out_of_range("x"); }  
catch (const std::logic_error& e) { cout << "logic"; }  
catch (const std::exception& e) { cout << "ex"; }
```

- (a) logic (out_of_range jaʻrangum logic_error)
- (b) ex
- (c) erku-n
- (d) terminate

C6

```
try { throw std::runtime_error("x"); }  
catch (const std::exception& e) { cout << "ex "; }  
catch (const std::runtime_error& e) { cout << "rt"; }
```

- (a) rt
- (b) ex (arajin hamapatasxan, verevic)

- (c) erku-n
- (d) CE

C7

```
struct A { ~A() { throw 1; } }; // dtor throw
void f() { A a; throw 2; }      // erkrord throw unwinding-i zham
```

- (a) caught
- (b) std::terminate (2 throw miasin)
- (c) OK
- (d) 1

C8

```
std::vector<int> v(3);
try { cout << v.at(10); }
catch (const std::out_of_range& e) { cout << "oor"; }
```

- (a) garbage
- (b) oor
- (c) UB
- (d) 0

C9

```
std::vector<int> v(3);
cout << v[10];          // operator[]
```

- (a) throw out_of_range
- (b) undefined behavior (operator[] chi stugum)
- (c) oor
- (d) 0

C10

```
void f() {  
    try { throw std::runtime_error("x"); }  
    catch (const std::exception& e) { throw; }    // rethrow  
}  
try { f(); } catch (const std::exception& e) { cout << "outer"; }
```

- (a) outer (rethrow -> durs propagate)
- (b) ochain
- (c) terminate
- (d) CE

C11

```
catch (std::exception e) { }    // by value
```

Derived exception-i depqum? - (a) OK, amboxj - (b) slicing (derived masy ktrvum) - (c) CE - (d) leak

C12

```
int* p = new int[10000000000000L];
```

Anhajoxutyan depqum? - (a) nullptr - (b) throw std::bad_alloc - (c) 0 - (d) crash aranc exception

C13

```
try { throw MyException("x"); }    // MyException : std::exception  
catch (const std::exception& e) { cout << e.what(); }
```

- (a) x (polymorphic what())
- (b) chi brnvi
- (c) CE
- (d) terminate

C14

```
struct A {  
    A() { throw std::runtime_error("ctor"); }  
    ~A() { cout << "~A"; }  
};  
try { A a; } catch(...) { cout << "caught"; }
```

- (a) ~A caught
- (b) caught (ctor throw -> object chi steghcvi -> dtor CHI kanchvi)
- (c) terminate
- (d) ~A

C15

```
void f() { std::string s = "hi"; throw 1; }  
try { f(); } catch(int) { cout << "caught"; }
```

s -i het inch? - (a) leak - (b) avtomat azatvum (stack unwinding -> s dtor) -> caught - (c) UB - (d) terminate

ANSWER KEY A

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b 11-b · 12-b · 13-b · 14-a · 15-b · 16-b · 17-b · 18-a · 19-b · 20-b 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

ANSWER KEY B (hakirch)

1. Error handling mekanizm; throw/try/catch; normal hosqy u error handling aʻrancin, avtomat propagate.
2. throw exception nety; try pahpanvac seksia; catch bʻrnum tesakov (verevic var).
3. throw-i zham C++-y het e gnum call stack-ov catch gtnele; ayd entacqum local object-neri dtor-ner kanchvum.
4. Stack unwinding -> RAI object-neri dtor-ner avtomat kanchvum -> resource cleanup (chka leak).

5. By value `slicing (derived ktrvum) + copy`. By reference `(const&)` polymorphic, chka slicing (CHISHT).
 6. Unwinding-i zham (arden meky throw) erkrord throw -> `std::terminate`. dtor petq noexcept.
 7. `std::exception` (`what()`); `logic_error` (`invalid_argument`, `out_of_range`), `runtime_error` (`overflow`), `bad_alloc`, `bad_cast`.
 8. `virtual const char* what() const noexcept`; error message.
 9. Jařangel `std::exception`, override `what()` vor veradarcni message.
 10. No-throw (noexcept), strong (rollback), basic (chka leak), none. RAII -> arřnvazn basic.
 11. Xostum vor function chi throw; compiler optimization + STL move; throw anelis -> terminate.
 12. move ctor noexcept linelu depqum STL (vector resize) ogtagorcum e move (ne copy) -> arag + strong guarantee.
 13. `catch(...)` amen inch brřnum; erb tipy anhayt e kam catch-all cleanup petq.
 14. Verevic var; arřajin hamapatasxan catch-y (dra hamar konkretic general).
 15. throw; (arřanc argument) catch-i mej -> nuyn exception-y durs propagate.
 16. Exception avtomat propagate, chi karoř lřeli anteswel; error code dzeřgov amen angam stugum (hesht mořanal).
 17. Zero-cost model` chka throw -> grethe zero overhead; throw-i zham tařk (bayc hazvadep).
 18. `bad_alloc` (new fail), `bad_cast` (`dynamic_cast` reference fail), `out_of_range` (`at()`).
 19. `logic_error` `kanxateseli programming sxal (invalid arg)`; `runtime_error` ashxatanki zham (I/O, overflow).
 20. ctor throw -> object chi hamarvum steghcvac -> ir dtor CHI kanchvi (bayc arden steghcvac member-neriny` ayo).
 21. Danak e (dandaghn throw-i zham, kod pakas pařz); normal flow-i hamar return/optional aveli lav.
 22. Exception neutrality` function-y brřnac exception-nery chi "kul talis", tuyl talis propagate (kam rethrow).
 23. Zero-cost` chka throw -> ~0 overhead (modern compiler); throw path-y dandaghn.
 24. Exception-ner chen propagate thread-neri mijev; ete thread-ic exception durs ga -> `std::terminate`. `std::exception_ptr`-ov karas poxancel.
 25. Function-try-block `constructor-i initializer list-i exception brřnelu hamar ( try` function-i mbody-ic durs)`.
-

ANSWER KEY C

C1-a · C2-b · C3-b · C4-b · C5-a · C6-b · C7-b · C8-b · C9-b · C10-a C11-b · C12-b · C13-a · C14-b · C15-b

Bacatrutyunner (C)

C1 runtime_error jaʻangum std::exception -> caught. **C2** throw int -> chka int catch (miayn exception&) -> terminate. **C3** stack unwinding -> A::~~A kanchvum -> "~A caught". **C4** noexcept + throw -> std::terminate. **C5** out_of_range jaʻangum logic_error -> "logic". **C6** verevic var, aʻajin hamapatasxan (exception&) -> "ex". **C7** dtor throw unwinding-i zham (arden throw 2) -> 2 throw -> terminate. **C8** at() -> throw out_of_range -> "oor". **C9** operator[] chi stugum -> UB. **C10** rethrow (throw;) -> durs propagate -> "outer". **C11** by value -> slicing. **C12** new fail -> bad_alloc. **C13** MyException : std::exception, polymorphic what() -> "x". **C14** ctor throw -> object chi steghcvi -> dtor CHI kanchvi -> "caught". **C15** stack unwinding -> s (string) dtor avtomat -> chka leak -> "caught".

Endhanur kanonner

```
throw/try/catch | catch verevic var (konkretic general) | catch by REFERENCE
stack unwinding -> local dtor-ner kanchvum (RAII cleanup)
dtor-ic CHTHROW (terminate) | noexcept+throw -> terminate
int throw -> chka exception& catch -> terminate
at() -> out_of_range | operator[] -> UB (chi stugum)
ctor throw -> object chi steghcvi (dtor chi kanchvi)
new fail -> bad_alloc | dynamic_cast ref fail -> bad_cast
custom: jaʻangel std::exception + override what()
rethrow: throw; (aʻanc argument)
```